Lab S11. Una verdad, varias copias

Kevin Santiago González | Grupo 2
Sistemas Distribuidos 2027-1
7 de noviembre de 2026

1. El punto de partida

Al levantar el banco, mongo y rabbitmq no arrancaron. Ninguno de los dos errores venía del código del lab.

Contenedor	Qué pasó	Arreglo
mongo	MongoDB 8 no arranca con el kernel 6.19 que trae ahora Docker Desktop (SERVER-121912).	Agregué la variable GLIBC_TUNABLES al servicio en docker-compose.yml.
rabbitmq	No podía leer .erlang.cookie (eacces). El volumen era de un lab anterior.	Borré el contenedor con su volumen: docker compose rm -s -f -v rabbitmq.

```
mongo:
  image: mongo:8.0
  environment:
    GLIBC_TUNABLES: glibc.pthread.rseq=1
```

Por ese cambio entrego también docker-compose.yml. Después corrí docker compose down -v para empezar con datos limpios. Redis contestó PONG y en Mongo había 0 avisos.

```
[docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments()'
PONG
0
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

Redis vivo y la colección avisos vacía.

Creé a Ana, Beto y Carla con 1000, 500 y 300. La primera transferencia de 200 respondió 503: movimiento acababa de arrancar y no contestó en los 2 s que espera la saga para registrar los asientos. Eso pasa después del abono, así que el dinero sí se movió y la transferencia 1 quedó en ABONADA. La repetí con los servicios ya calientes y quedó COMPLETADA.

Cuenta	Inicial	Después de las transferencias 1 y 2
Ana: 002180000000000001	1000.00	600.00
Beto: 002180000000000002	500.00	900.00
Carla: 002180000000000003	300.00	300.00

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"00218000000000001","titular":"Ana","saldoInicial":1000}'
curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"00218000000000002","titular":"Beto","saldoInicial":500}'
curl -i -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' \
  -d '{"clabe":"00218000000000003","titular":"Carla","saldoInicial":300}'

curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":200}'; echo
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 834f527f3ceb
Content-Type: application/json
Date: Sat, 07 Nov 2026 06:07:32 GMT

{"id":1,"clabe":"00218000000000001","titular":"Ana","saldo":1000}HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 74b34d8201a1
Content-Type: application/json
Date: Sat, 07 Nov 2026 06:07:33 GMT

{"id":2,"clabe":"00218000000000002","titular":"Beto","saldo":500}HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 79dd39dd1948
Content-Type: application/json
Date: Sat, 07 Nov 2026 06:07:34 GMT

{"id":3,"clabe":"00218000000000003","titular":"Carla","saldo":300}{"type":"about:blank","title":"Service Unavailable","status":503,"detail":"No se pudo contactar a otro servicio: I/O error on POST request for \"http://movimiento/movimientos\": Read timed out","instance":"/transferencias"}
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % curl -s http://localhost:8080/api/transferencias/1; echo
curl -s http://localhost:8080/api/cuentas/00218000000000001; echo
curl -s http://localhost:8080/api/cuentas/00218000000000002; echo
{"id":1,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":200.00,"fecha":"2026-11-07T06:07:36.797901Z","estado":"ABONADA","motivo":null,"saldoOrigenTrasCargo":800.00,"intentosDeCompensacion":0,"actualizada":"2026-11-07T06:07:37.748059Z","eventoId":"1095b1c5-d181-484e-b2fa-0ca0d574f06c"}
{"clabe":"00218000000000001","titular":"Ana","saldo":800.00,"leidaEn":"2026-11-07T06:09:03.995595751Z"}
{"clabe":"00218000000000002","titular":"Beto","saldo":700.00,"leidaEn":"2026-11-07T06:09:04.189219959Z"}
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":200}'; echo
{"id":2,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":200,"fecha":"2026-11-07T06:09:24.023051427Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":600.00,"intentosDeCompensacion":0,"actualizada":"2026-11-07T06:09:25.000257636Z","eventoId":"36eabd74-6c1d-4a93-91ec-8c7f01c695b6"}
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

Altas con 201, transferencia 1 en ABONADA y transferencia 2 COMPLETADA.

Antes de tocar código medí la consulta del saldo. Pedí el saldo de Ana diez veces y cada respuesta trajo un leidaEn distinto. El log de cuenta tenía 16 lecturas a Postgres: las diez consultas más las de las transferencias. Redis marcó 0 hits y 0 misses.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % for i in $(seq 1 10); do
  curl -s http://localhost:8080/api/cuentas/00218000000000001; echo
done

docker compose logs cuenta | grep -c "leida de Postgres"

docker compose exec -T redis redis-cli INFO stats | grep keyspace
{"clabe":"00218000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:11:13.703718047Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:11:13.881954797Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:11:14.035266506Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:11:14.075574297Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:11:14.122800006Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:11:14.162515547Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:11:14.209392506Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:11:14.248950672Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:11:14.284116964Z"}
{"clabe":"00218000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:11:14.326422631Z"}
16
keyspace_hits:0
keyspace_misses:0
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

Antes de la caché: cada consulta se leyó de Postgres.

2. La caché

Usé cache-aside. La consulta que muestra el saldo pregunta primero a Redis; si no encuentra la copia, lee Postgres y la guarda. Los cargos y los abonos la borran.

En ConfiguracionCache agregué @EnableCaching y un bean para la caché cuentas. Cada entrada vive 30 s, el valor se guarda como JSON de CuentaVista y el borrado espera al commit.

```
@EnableCaching
@Configuration
public class ConfiguracionCache {
    @Bean
    RedisCacheManagerBuilderCustomizer cacheDeCuentas(
            @Value("${mexibanco.cache.ttl:30s}") Duration ttl, ObjectMapper json) {
        RedisCacheConfiguration cuentas = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(ttl)
            .disableCachingNullValues()
            .serializeValuesWith(SerializationPair.fromSerializer(
                new Jackson2JsonRedisSerializer<>(json, CuentaVista.class)));
        return builder -> builder.withCacheConfiguration(CUENTAS, cuentas)
            .transactionAware();
    }
}
```

Puse @Cacheable en ConsultaDeSaldo.consultar y @CacheEvict en CuentaService.cargar y abonar, las dos con la CLABE como clave.

```
@Cacheable(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
public CuentaVista consultar(String clabe) { ... }

@CacheEvict(cacheNames = ConfiguracionCache.CUENTAS, key = "#clabe")
@Transactional
public SaldoAplicado cargar(String clabe, BigDecimal monto, String clave) { ... }
```

No toqué CuentaService.consultar. Es la lectura con la que cargar decide si alcanza el saldo y tiene que ir siempre a Postgres. Por eso la consulta que muestra está en otra clase; además, un @Cacheable llamado desde la misma clase no pasa por el proxy de Spring.

Reconstruí cuenta y pedí el saldo de Ana seis veces. Contestaron tres instancias distintas, pero las seis respuestas trajeron el mismo leidaEn.

Prueba	Consultas	Lecturas a Postgres
Sin caché	10	16 (con las de las transferencias)
Con caché	6	1

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % for i in 1 2 3 4 5 6; do
  curl -s -D - http://localhost:8080/api/cuentas/002180000000000001 | grep -iE 'x-instancia|leidaEn'
done

docker compose logs --since 30s cuenta | grep -c "leida de Postgres"

docker compose exec -T redis redis-cli GET cuentas::002180000000000001
docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
X-Instancia: fd6de49b7aeb
{"clabe":"002180000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:15:16.783331799Z"}
X-Instancia: a4048828b968
{"clabe":"002180000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:15:16.783331799Z"}
X-Instancia: 5ecf0e9c2f87
{"clabe":"002180000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:15:16.783331799Z"}
X-Instancia: fd6de49b7aeb
{"clabe":"002180000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:15:16.783331799Z"}
X-Instancia: a4048828b968
{"clabe":"002180000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:15:16.783331799Z"}
X-Instancia: 5ecf0e9c2f87
{"clabe":"002180000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:15:16.783331799Z"}
1
{"clabe":"002180000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:15:16.783331799Z"}
27
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

Las tres copias de cuenta leen la misma copia en Redis, con su JSON y TTL 27.

Después dejé redis-cli MONITOR abierto y transferí 10 de Ana a Beto. Al terminar cada commit llegó un DEL: el de Ana y el de Beto vinieron de dos copias distintas de cuenta. La siguiente consulta fue un miss y volvió a guardar la copia con el saldo nuevo y PX 30000.

```
Last login: Fri Nov  6 23:48:10 on ttys000
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % cd mexi-banco-s11
docker compose exec -T redis redis-cli MONITOR
cd: no such file or directory: mexi-banco-s11
OK
1794032212.791570 [0 127.0.0.1:34756] "ping"
1794032217.830686 [0 127.0.0.1:34768] "ping"
1794032222.868076 [0 127.0.0.1:41684] "ping"
1794032227.927845 [0 127.0.0.1:41694] "ping"
1794032230.489236 [0 172.19.0.11:52290] "GET" "cuentas::002180000000000001"
1794032230.550526 [0 172.19.0.11:52290] "SET" "cuentas::002180000000000001" "{\"clabe\":\"002180000000000001\",\"titular\":\"Ana\",\"saldo\":600.00,\"leidaEn\":\"2026-11-07T06:17:10.535858754Z\"}" "PX"
"30000"
1794032230.735957 [0 172.19.0.14:55926] "GET" "cuentas::002180000000000001"
1794032230.780741 [0 172.19.0.13:38896] "GET" "cuentas::002180000000000002"
1794032231.149753 [0 172.19.0.13:38896] "SET" "cuentas::002180000000000002" "{\"clabe\":\"002180000000000002\",\"titular\":\"Beto\",\"saldo\":900.00,\"leidaEn\":\"2026-11-07T06:17:11.133813296Z\"}" "PX"
 "30000"
1794032231.547075 [0 172.19.0.11:52290] "DEL" "cuentas::002180000000000001"
1794032231.968906 [0 172.19.0.14:55926] "DEL" "cuentas::002180000000000002"
1794032232.536109 [0 127.0.0.1:37428] "EXISTS" "cuentas::002180000000000001"
1794032232.703655 [0 172.19.0.13:38896] "GET" "cuentas::002180000000000001"
1794032232.736319 [0 172.19.0.13:38896] "SET" "cuentas::002180000000000001" "{\"clabe\":\"002180000000000001\",\"titular\":\"Ana\",\"saldo\":590.00,\"leidaEn\":\"2026-11-07T06:17:12.727381172Z\"}" "PX"
"30000"
1794032232.980442 [0 127.0.0.1:37442] "ping"
1794032238.026773 [0 127.0.0.1:37454] "ping"
1794032243.081620 [0 127.0.0.1:58562] "ping"
1794032248.137266 [0 127.0.0.1:58576] "ping"
```

MONITOR: DEL de Ana y de Beto después de la transferencia, y SET con 590.00.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % curl -s http://localhost:8080/api/cuentas/002180000000000001; echo

curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10}'; echo

docker compose exec -T redis redis-cli EXISTS cuentas::002180000000000001

curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
{"clabe":"002180000000000001","titular":"Ana","saldo":600.00,"leidaEn":"2026-11-07T06:17:10.535858754Z"}
{"id":3,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":10,"fecha":"2026-11-07T06:17:11.160131004Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":590.00,"inte
ntosDeCompensacion":0,"actualizada":"2026-11-07T06:17:12.310670505Z","eventoId":"cb2366b2-849e-4fad-ad20-2acd4b83fa88"}
0
{"clabe":"002180000000000001","titular":"Ana","saldo":590.00,"leidaEn":"2026-11-07T06:17:12.727381172Z"}
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

EXISTS dio 0 y la consulta trajo 590.00 con un leidaEn nuevo.

3. Mis medidas

Medí el hit ratio con 100 consultas seguidas (A) y con una transferencia de 1 peso cada diez consultas (B).

Escenario	Hits	Misses	Hit ratio
A: 100 consultas seguidas	99	1	99 %
B: una transferencia cada 10 consultas	101	19	84.2 %

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % docker compose exec -T redis redis-cli CONFIG RESETSTAT
for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
done
docker compose exec -T redis redis-cli INFO stats | grep keyspace
OK
keyspace_hits:99
keyspace_misses:1
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

Escenario A: 99 hits y 1 miss.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % docker compose exec -T redis redis-cli CONFIG RESETSTAT
for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
  if [ $((i % 10)) -eq 0 ]; then
    curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
      -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":1}'
  fi
done
docker compose exec -T redis redis-cli INFO stats | grep keyspace
OK
keyspace_hits:101
keyspace_misses:19
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

Escenario B: 101 hits y 19 misses.

En B hubo 120 lecturas porque cada transferencia consulta a Ana y a Beto antes de mover dinero. Cada transferencia borró las dos copias: la comprobación de Beto falló diez veces y mi siguiente consulta de Ana falló nueve.

Después medí la latencia de 100 consultas, tres veces con caché y tres sin ella. Para apagarla no cambié código: levanté cuenta con CACHE_TIPO=none e hice 30 consultas de calentamiento antes de medir.

Corrida	Caché	Promedio (ms)	Mediana (ms)	p95 (ms)	Lecturas a Postgres
1	con	13.85	7.19	22.84	1
2	con	22.62	11.68	57.54	0
3	con	8.69	6.24	14.56	0
1	sin	8.72	8.00	13.21	127
2	sin	12.20	9.57	21.41	100
3	sin	8.85	7.58	14.56	100

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % medir; medir; medir
promedio=13.85 ms mediana=7.19 ms p95=22.84 ms
lecturas a Postgres: 1
promedio=22.62 ms mediana=11.68 ms p95=57.54 ms
lecturas a Postgres: 0
promedio=8.69 ms mediana=6.24 ms p95=14.56 ms
lecturas a Postgres: 0
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

Tres corridas con caché.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % for i in $(seq 1 30); do curl -s -o /dev/null http://localhost:8080/api/cuentas/00218000000000001; done
medir; medir; medir
promedio=8.72 ms mediana=8.00 ms p95=13.21 ms
lecturas a Postgres: 127
promedio=12.20 ms mediana=9.57 ms p95=21.41 ms
lecturas a Postgres: 100
promedio=8.85 ms mediana=7.58 ms p95=14.56 ms
lecturas a Postgres: 100
```

Tres corridas sin caché.

La primera corrida sin caché contó 127 lecturas porque 27 consultas del calentamiento cayeron en el mismo segundo en que empezó el conteo. Las medianas quedaron entre 6 y 12 ms en los dos casos; la diferencia es menor que la variación entre corridas. Lo que cambió fue Postgres: de 100 lecturas por corrida a 0 o 1.

4. Los avisos

Cada aviso es un documento en la colección avisos. El _id lo arma el banco con eventoId:clabe. Los campos comunes van en todos y después un switch sobre Contacto agrega los del canal. El dinero va en Decimal128.

```
Document aviso = new Document("_id", evento.eventoId() + ":" + destinatario.clabe())
    .append("eventoId", evento.eventoId().toString())
    .append("transferenciaId", evento.transferenciaId())
    .append("clabe", destinatario.clabe())
    .append("tipo", destinatario.tipo())
    .append("monto", new Decimal128(evento.monto()))
    .append("saldo", new Decimal128(destinatario.saldo()))
    .append("fecha", Date.from(evento.fecha()));
switch (contacto) {
    case Contacto.Sms sms -> aviso.append("canal", "SMS")
        .append("telefono", sms.telefono()).append("texto", texto);
    case Contacto.Correo correo -> aviso.append("canal", "CORREO")
        .append("correo", correo.correo()).append("asunto", encabezado(destinatario))
        .append("cuerpo", texto);
    case Contacto.Push push -> aviso.append("canal", "PUSH")
        .append("token", push.token()).append("titulo", encabezado(destinatario))
        .append("cuerpo", texto);
}
```

registrar inserta los avisos uno por uno. Si Mongo lanza DuplicateKeyException, el aviso ya estaba: no lo cuento y sigo con el siguiente. Los inserto por separado porque un insert de varios se detiene en el primer duplicado.

```
try {
    mongo.insert(aviso, COLECCION);
    nuevos++;
} catch (DuplicateKeyException yaEstaba) {
    log.info("[avisos] {} ya estaba: evento repetido, no se duplica", aviso.get("_id"));
}
```

En ConsumidorDeTransferencias cambié una línea: nuevos = bitacora.registrar(evento). Los avisos se mudan a Mongo. No los guardo también en Postgres, porque sería una doble escritura sin transacción común.

Reconstruí notificacion. Al arrancar creó el índice clabe_fecha, con clabe ascendente y fecha descendente.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % docker compose logs notificacion | grep avisos
notificacion-1  | 2026-11-07T06:25:26.397Z  INFO 1 --- [notificacion] [           main] m.m.n.avisos.BitacoraDeAvisos           : [avisos] indice clabe_fecha listo en avisos
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

notificacion creó el índice clabe_fecha al arrancar.

Las tres formas

Hice transferencias de Ana a Beto y de Ana a Carla. Ana recibe SMS, Beto correo y Carla push.

Canal	Cuenta	Campos propios
SMS	Ana	telefono, texto
CORREO	Beto	correo, asunto, cuerpo
PUSH	Carla	token, titulo, cuerpo

```
{
  _id: '110ae71a-40f5-4c37-b743-aa6300ba6192:002180000000000001',
  eventoId: '110ae71a-40f5-4c37-b743-aa6300ba6192',
  transferenciaId: Long('14'),
  clabe: '002180000000000001',
  tipo: 'DEVOLUCION_TRANSFERENCIA',
  monto: Decimal128('50'),
  saldo: Decimal128('580.00'),
  fecha: ISODate('2026-11-07T06:26:42.613Z'),
  canal: 'SMS',
  telefono: '+52 55 5555 0101',
  texto: 'DEVOLUCION_TRANSFERENCIA por 50, saldo resultante 580.00'
},
{
  _id: '0722151b-123a-44a3-bf39-f04f329dda93:002180000000000001',
  eventoId: '0722151b-123a-44a3-bf39-f04f329dda93',
  transferenciaId: Long('15'),
  clabe: '002180000000000001',
  tipo: 'TRANSFERENCIA_ENVIADA',
  monto: Decimal128('50'),
  saldo: Decimal128('530.00'),
  fecha: ISODate('2026-11-07T06:26:43.154Z'),
  canal: 'SMS',
  telefono: '+52 55 5555 0101',
  texto: 'TRANSFERENCIA_ENVIADA por 50, saldo resultante 530.00'
},
{
  _id: '0722151b-123a-44a3-bf39-f04f329dda93:002180000000000003',
  eventoId: '0722151b-123a-44a3-bf39-f04f329dda93',
  transferenciaId: Long('15'),
  clabe: '002180000000000003',
  tipo: 'TRANSFERENCIA_RECIBIDA',
  monto: Decimal128('50'),
  saldo: Decimal128('350.00'),
  fecha: ISODate('2026-11-07T06:26:43.154Z'),
  canal: 'PUSH',
  token: 'fcm:demo-002180000000000003',
  titulo: 'Recibiste una transferencia',
  cuerpo: 'TRANSFERENCIA_RECIBIDA por 50, saldo resultante 350.00'
},
{
  _id: 'a2b56efd-6a14-4474-b9c3-69e95295d844:002180000000000001',
  eventoId: 'a2b56efd-6a14-4474-b9c3-69e95295d844',
  transferenciaId: Long('16'),
  clabe: '002180000000000001',
  tipo: 'TRANSFERENCIA_ENVIADA',
  monto: Decimal128('50'),
  saldo: Decimal128('480.00'),
  fecha: ISODate('2026-11-07T06:31:00.706Z'),
  canal: 'SMS',
  telefono: '+52 55 5555 0101',
  texto: 'TRANSFERENCIA_ENVIADA por 50, saldo resultante 480.00'
},
{
  _id: 'a2b56efd-6a14-4474-b9c3-69e95295d844:002180000000000002',
  eventoId: 'a2b56efd-6a14-4474-b9c3-69e95295d844',
  transferenciaId: Long('16'),
  clabe: '002180000000000002',
  tipo: 'TRANSFERENCIA_RECIBIDA',
```

```
{
  _id: 'a2b56efd-6a14-4474-b9c3-69e95295d844:002180000000000002',
  eventoId: 'a2b56efd-6a14-4474-b9c3-69e95295d844',
  transferenciaId: Long('16'),
  clabe: '002180000000000002',
  tipo: 'TRANSFERENCIA_RECIBIDA',
  monto: Decimal128('50'),
  saldo: Decimal128('1020.00'),
  fecha: ISODate('2026-11-07T06:31:00.706Z'),
  canal: 'CORREO',
  correo: 'beto@correo.mx',
  asunto: 'Recibiste una transferencia',
  cuerpo: 'TRANSFERENCIA_RECIBIDA por 50, saldo resultante 1020.00'
}

kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

Los avisos guardados, del más viejo al más nuevo. A la derecha, el último: el de Beto, con canal CORREO.

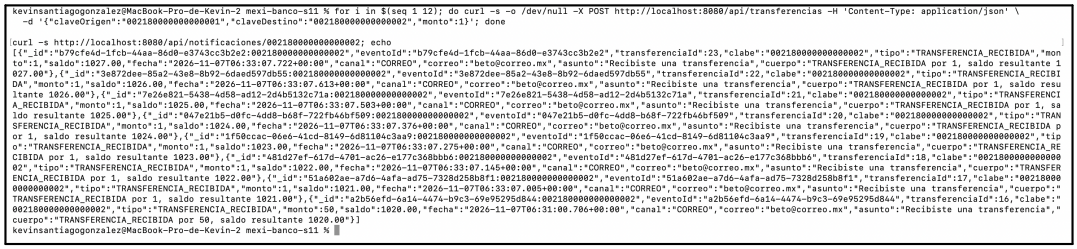
El primer aviso es una DEVOLUCION_TRANSFERENCIA de la transferencia 14; lo explico al final de esta sección. La consulta por telefono trajo solo los tres avisos de Ana: el esquema lo aplica quien lee.

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists: true}}, {clabe: 1, canal: 1, telefono: 1})'
[
  {
    _id: '110ae71a-40f5-4c37-b743-aa6300ba6192:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: '0722151b-123a-44a3-bf39-f04f329dda93:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: 'a2b56efd-6a14-4474-b9c3-69e95295d844:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

Solo los avisos de Ana tienen el campo telefono.

Los últimos 10 y el índice

Hice doce transferencias de 1 peso de Ana a Beto y pedí la bitácora de Beto en cuanto terminaron. Solo llegaron 8 avisos. Cada transferencia responde cuando guarda su evento en el outbox, y el aviso llega después por RabbitMQ.



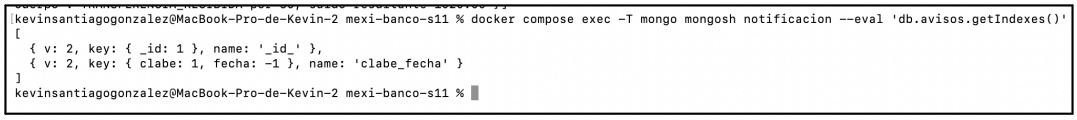
La bitácora de Beto justo al terminar: todavía faltaban avisos.

Unos segundos después repetí la consulta y llegaron exactamente 10, todos CORREO, de la transferencia 28 a la 19.



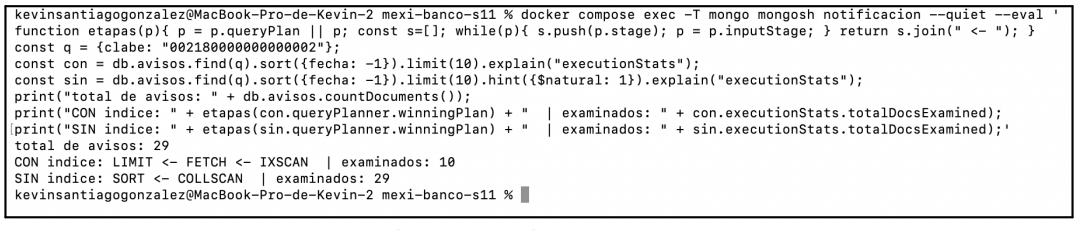
Los últimos 10 avisos de Beto, del más nuevo al más viejo.

getIndexes mostró _id_ y clabe_fecha. Después corrí la consulta de ultimos con explain, con el índice y obligando a Mongo a recorrer la colección con hint({$natural: 1}). Con 29 avisos la diferencia es chica; con millones, la consulta con índice sigue leyendo 10 documentos y la otra los lee todos.



Índices de la colección avisos.

Plan	Etapas	Documentos examinados
Con índice	LIMIT ← FETCH ← IXSCAN	10
Sin índice	SORT ← COLLSCAN	29, todos



Plan con índice y sin índice sobre 29 avisos.

Un timeout que creó 50 pesos

Después de volver a encender la caché transferí 50 de Ana a Beto. cuenta acababa de reiniciar y el abono tardó más de los 2 s que espera la saga. La saga lo tomó como fallido, devolvió los 50 a Ana y dejó la transferencia 14 en COMPENSADA.

Pero el abono sí se había aplicado. En operacion_aplicada quedaron el cargo, la devolución y el abono a Beto.

Cuenta	Esperado	Real
Ana	530.00	530.00
Beto	920.00	970.00

```
       clabe         | titular | saldo
---------------------+---------+--------
 002180000000000001 | Ana     | 530.00
 002180000000000002 | Beto    | 970.00
 002180000000000003 | Carla   | 350.00
(3 rows)

            clave             |       clabe        | tipo  | monto | saldo_resultante
------------------------------+--------------------+-------+-------+------------------
 transferencia-14-cargo       | 002180000000000001 | CARGO | 50.00 |           530.00
 transferencia-14-devolucion  | 002180000000000001 | ABONO | 50.00 |           580.00
 transferencia-14-abono       | 002180000000000002 | ABONO | 50.00 |           970.00
(3 rows)
```

Saldos en Postgres y operaciones de la transferencia 14.

No lo causó la caché. La saga de la S08 compensa cuando no recibe respuesta, pero un timeout de lectura no prueba que la operación no ocurrió. Antes de devolver el dinero tendría que preguntar a cuenta por la clave transferencia-14-abono. No corregí el saldo a mano; lo dejé como evidencia.

5. El duplicado y el dato viejo

El evento duplicado

Publiqué dos veces el mismo evento desde la API de RabbitMQ, con eventoId 11111111-2222-4333-8444-555555555555 y CLABEs inventadas que terminan en 091 y 092. Las dos publicaciones respondieron routed true.

Publicaciones	Avisos si se duplicaran	Documentos en Mongo
2	4	2

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % for i in 1 2; do
curl -s -u mexibanco:mexibanco -X POST http://localhost:15672/api/exchanges/%2F/transferencias/publish \
  -H 'Content-Type: application/json' \
  -d '{"properties":{"delivery_mode":2,"content_type":"application/json"},"routing_key":"transferencia.realizada","payload_encoding":"string","payload":"{\"eventoId\":\"11111111-2222-4333-8444-555555555
555\",\"tipo\":\"TransferenciaRealizada\",\"transferenciaId\":9001,\"clabeOrigen\":\"002180000000000091\",\"clabeDestino\":\"002180000000000092\",\"monto\":25,\"saldoOrigen\":475,\"saldoDestino\":525,\"
fecha\":\"2026-10-31T13:00:00Z\"}"}'
echo
done

sleep 2
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments({eventoId: "11111111-2222-4333-8444-555555555555"})'
docker compose logs notificacion | grep "ya estaba"
{"routed":true}
{"routed":true}
2
notificacion-1  | 2026-11-07T06:36:42.723Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos            : [avisos] 11111111-2222-4333-8444-555555555555:002180000000000091 ya est
aba: evento repetido, no se duplica
notificacion-1  | 2026-11-07T06:36:42.730Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos            : [avisos] 11111111-2222-4333-8444-555555555555:002180000000000092 ya est
aba: evento repetido, no se duplica
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

El mismo evento dos veces dejó 2 documentos y dos registros de ya estaba.

El segundo insert de cada aviso chocó con el mismo _id. No necesité la tabla evento_procesado de la S09: la clave primaria del documento hace el mismo trabajo.

El dato viejo

Pedí el saldo de Ana para dejarlo en la caché y cambié su saldo a 5 directo en Postgres. Ese update no pasa por @CacheEvict, así que nadie borró la copia.

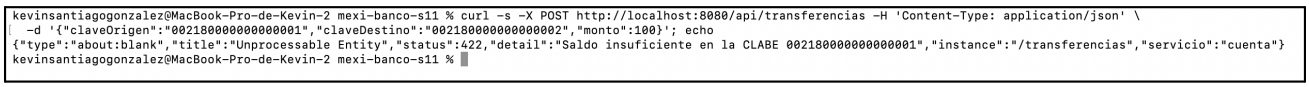
En el primer intento pasaron 35 s entre la consulta y el update; la copia ya había caducado y no se vio el dato viejo. Lo repetí con todos los comandos seguidos.

TTL	Saldo que vio el cliente	Saldo en Postgres
30	468.00	5.00
18	468.00	5.00
2	468.00	5.00
-2	5.00	5.00

```
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 % docker compose exec -T db psql -U mexibanco -d cuenta -c "update cuenta set saldo = 468 where clabe = '002180000000000001'"
docker compose exec -T redis redis-cli DEL cuentas::002180000000000001
curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
docker compose exec -T db psql -U mexibanco -d cuenta -c "update cuenta set saldo = 5 where clabe = '002180000000000001'"
for i in $(seq 1 8); do
  docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
  curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
  sleep 5
done
UPDATE 1
0
{"clabe":"002180000000000001","titular":"Ana","saldo":468.00,"leidaEn":"2026-11-07T06:41:22.511659176Z"}
UPDATE 1
30
{"clabe":"002180000000000001","titular":"Ana","saldo":468.00,"leidaEn":"2026-11-07T06:41:22.511659176Z"}
24
{"clabe":"002180000000000001","titular":"Ana","saldo":468.00,"leidaEn":"2026-11-07T06:41:22.511659176Z"}
18
{"clabe":"002180000000000001","titular":"Ana","saldo":468.00,"leidaEn":"2026-11-07T06:41:22.511659176Z"}
13
{"clabe":"002180000000000001","titular":"Ana","saldo":468.00,"leidaEn":"2026-11-07T06:41:22.511659176Z"}
8
{"clabe":"002180000000000001","titular":"Ana","saldo":468.00,"leidaEn":"2026-11-07T06:41:22.511659176Z"}
2
{"clabe":"002180000000000001","titular":"Ana","saldo":468.00,"leidaEn":"2026-11-07T06:41:22.511659176Z"}
-2
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-07T06:41:55.904615678Z"}
25
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leidaEn":"2026-11-07T06:41:55.904615678Z"}
kevinsantiagogonzalez@MacBook-Pro-de-Kevin-2 mexi-banco-s11 %
```

El cliente vio 468.00 hasta que caducó la copia.

La app mostró 468.00 unos 30 s, siempre con el mismo leidaEn. Mientras tanto intenté transferir 100 de Ana a Beto y respondió 422 por saldo insuficiente, porque cargar lee Postgres. Al terminar regresé el saldo de Ana a 468.



La transferencia de 100 fue rechazada mientras la pantalla mostraba 468.00.

Lo que comprobé

Prueba	Resultado
Caché compartida	Tres instancias contestaron con una sola lectura a Postgres.
Cargo y abono	DEL después del commit y saldo nuevo en la siguiente consulta.
Hit ratio	99 % en A y 84.2 % en B.
Latencia	Casi igual; Postgres pasó de 100 lecturas a 0 o 1.
Avisos	Tres formas, 10 en el endpoint y 10 documentos examinados con índice.
Evento repetido	Quedaron 2 documentos en lugar de 4.
Dato viejo	468.00 visible unos 30 s; la transferencia de 100 fue rechazada.

Hice las pruebas manuales mostradas. La construcción Docker omitió las pruebas automáticas; no presento la suite como ejecutada.

6. Preguntas de cierre

Por qué MongoDB para los avisos y no para el saldo

Cada aviso tiene la forma de su canal: el SMS lleva telefono, el correo lleva correo y asunto, y el push lleva token y titulo. Se escribe una vez, no se actualiza y se lee entero por CLABE. En Mongo es un documento por aviso, sin columnas vacías, y el índice clabe_fecha resuelve la única consulta. El saldo necesita una transacción: el cargo revisa y resta junto con su Idempotency-Key, con @Version contra escrituras concurrentes, y cuadra con los movimientos en Postgres.

Qué hit ratio medí y qué compré

Medí 99 % en A y 84.2 % en B. B baja porque cada transferencia borra las copias de Ana y de Beto y la siguiente lectura de cada uno falla. La mediana quedó entre 6 y 12 ms con y sin caché, pero Postgres pasó de 100 lecturas por corrida a 0 o 1. Compré capacidad, no velocidad: pesa más el camino por el gateway que leer una fila por su llave.

Si el cargo leyera la caché

En la Parte 4 la caché mostró 468.00 durante unos 30 s mientras Postgres tenía 5. Si cargar decidiera con esa copia, habría aprobado la transferencia de 100 y Ana habría quedado en -95. Como lee Postgres, respondió 422. La caché muestra; no decide.

Por qué DEL y no SET, y por qué después del commit

Con SET, dos cambios concurrentes pueden escribir en Redis en otro orden que sus commits y dejar el saldo viejo como último valor. Un SET dentro de una transacción que luego hace rollback guardaría un saldo que nunca existió. DEL no depende del orden: deja la caché vacía y la siguiente consulta la llena desde Postgres. Si borrara antes del commit, una consulta en esa ventana leería el saldo anterior y lo volvería a guardar por 30 s. transactionAware() hace que el DEL espere.

Lo que elegí y lo que sacrifiqué

Elección: copiar en Redis el saldo que se muestra, con TTL de 30 s, y borrarlo en cada cargo y abono.
Porque: las consultas del saldo dejan de cargar a Postgres.
Sacrifico: el saldo que se muestra puede ir hasta 30 s atrás si alguien lo cambia por fuera de la app.

Lo que todavía falta

La saga sigue compensando abonos tardíos aunque ya se hayan aplicado, y GLIBC_TUNABLES es solo un parche para el kernel de Docker Desktop.

7. Redis caído

No hice el reto opcional. Sin él, un error de Redis no está manejado y puede tumbar la consulta del saldo; no lo probé.

Referencias

Consigna del Lab S11, Sistemas Distribuidos 2027-1, grupo 2.
Spring Framework: docs.spring.io/spring-framework/reference/integration/cache.html.
Spring Data Redis: docs.spring.io/spring-data/redis/reference/redis/redis-cache.html.
MongoDB: mongodb.com/docs/manual/reference/explain-results/.
MongoDB y el kernel 6.19: jira.mongodb.org/browse/SERVER-121912.